



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Selected Topics in Cryptography

Reporte Práctica 05:

“ECDH with standard elliptic curves”

Alumno:

Sigala Morales Said

2022630413

Grupo: 7CM4

Maestro: Sandra Diaz Santiago

Fecha de realización: 02 de abril de 2025

Fecha de entrega: 02 de abril de 2025



PROGRAMMING EXERCISES

Función que me permite seleccionar la curva

```
1  def seleccionar_curva():
2
3      print("\nSeleccione la curva ECDH que desea usar:")
4      print("1) secp224r1 (NIST P-224)")
5      print("2) secp256r1 (NIST P-256)")
6      print("3) secp384r1 (NIST P-384)")
7      print("4) secp521r1 (NIST P-521)")
8      opcion = input("Opción: ")
9
10     if opcion == '1':
11         return ec.SECP224R1()
12     elif opcion == '2':
13         return ec.SECP256R1()
14     elif opcion == '3':
15         return ec.SECP384R1()
16     elif opcion == '4':
17         return ec.SECP521R1()
18     else:
19         print("Opción no válida. Se usará P-256 por defecto.")
20         return ec.SECP256R1()
21
```

Descripción de la función:

- Se imprimen en pantalla las opciones de curvas disponibles (cuatro curvas de NIST).
- Se lee la opción que el usuario ingresa (`input("Opción: ")`).
- Según la opción, se retorna el objeto de la curva correspondiente, por ejemplo `ec.SECP256R1()`.
- Si la opción no coincide con ninguna de las 4, se toma por defecto la curva P-256 (`ec.SECP256R1()`).

Esta función únicamente se encarga de que el usuario elija (o por defecto) la curva que se utilizará en ECDH (Diffie-Hellman de curva elíptica).



Función generar_par_llaves_priv_pub()

```
1 def generar_par_llaves_priv_pub():
2
3     curva = seleccionar_curva()
4
5     private_key = ec.generate_private_key(curva, default_backend())
6     public_key = private_key.public_key()
7
8     # Serializar llave privada en PEM (PKCS8)
9     private_pem = private_key.private_bytes(
10         encoding=serialization.Encoding.PEM,
11         format=serialization.PrivateFormat.PKCS8,
12         encryption_algorithm=serialization.NoEncryption()
13     )
14
15     # Serializar llave pública en PEM (SubjectPublicKeyInfo)
16     public_pem = public_key.public_bytes(
17         encoding=serialization.Encoding.PEM,
18         format=serialization.PublicFormat.SubjectPublicKeyInfo
19     )
20
21     # Guardar en archivos
22     with open("ecdh_priv.pem", 'wb') as f:
23         f.write(private_pem)
24     with open("ecdh_pub.pem", 'wb') as f:
25         f.write(public_pem)
26
27     print(f"\nPar de llaves ECDH generado y guardado:")
28     print(" - ecdh_priv.pem (llave privada, PEM)")
29     print(" - ecdh_pub.pem (llave pública, PEM)")
30     print(f"Curva utilizada: {curva.name}\n")
```

Descripción de la función:

- `curva = seleccionar_curva()`: Llama a la función anterior para obtener la curva elegida por el usuario.
- `private_key = ec.generate_private_key(curva, default_backend())`: Genera la llave privada ECDH a partir de la curva elegida.
- `public_key = private_key.public_key()`: De la llave privada se deriva la llave pública.
- Serialización de la llave privada:
`private_key.private_bytes(...)`: Se convierte la llave privada a bytes en formato PEM (PKCS8), sin encriptar (`NoEncryption()`).



- Serialización de la llave pública:
`public_key.public_bytes(...)`: Se convierte la llave pública a bytes en formato PEM (`SubjectPublicKeyInfo`).

Almacenamiento en archivos:

Se abren (o crean) dos archivos (`ecdh_priv.pem` y `ecdh_pub.pem`) en modo escritura binaria (`'wb'`) y se guardan las llaves.



ecdh_acordar_llave()

```
1 private_key = serialization.load_pem_private_key(
2     priv_pem,
3     password=None,
4     backend=default_backend()
5 )
6
7 print("\nIngresa la llave pública del otro (DER Base64):")
8 remote_pub_b64 = input("> ")
9
10 try:
11     remote_pub_der = base64.b64decode(remote_pub_b64.strip())
12 except Exception:
13     print("ERROR: no se pudo decodificar la Base64. Revisa la cadena.")
14     return
15
16 try:
17     remote_public_key = serialization.load_der_public_key(
18         remote_pub_der,
19         backend=default_backend()
20     )
21 except Exception:
22     print("ERROR: no se pudo cargar la llave pública DER.")
23     return
24
25 # Generar la clave ECDH (clave compartida)
26 shared_key = private_key.exchange(ec.ECDH(), remote_public_key) # bytes
27
28 # Mostrar en Base64
29 shared_b64 = base64.b64encode(shared_key).decode('utf-8')
30
31 print("\n=== CLAVE ECDH (COMPARTIDA) EN BASE64 ===")
32 print(shared_b64)
33 print("=====")
34
35 # Opcional: Guardar en un archivo
36 opcion = input("\n¿Deseas guardar esta clave ECDH en un archivo? (s/n): ")
37 if opcion.lower() == 's':
38     with open("shared_ecdh.key", 'w') as f:
39         f.write(shared_b64)
40     print("Archivo 'shared_ecdh.key' creado con éxito.")
```

Verificación de existencia de la llave privada: Comprueba si ecdh_priv.pem existe; de lo contrario, no puede continuar.



Carga de la llave privada: Se lee el contenido del archivo y se carga con `serialization.load_pem_private_key(...)`.

Solicitud de la llave pública remota (Base64): El usuario ingresa la llave pública del otro en formato DER pero codificada en Base64.

Decodificación: Se convierte la cadena Base64 a bytes DER (`base64.b64decode(...)`).

Reconstrucción de la llave pública remota: Se carga con `serialization.load_der_public_key(...)`.

Cálculo de la clave ECDH:

`private_key.exchange(ec.ECDH(), remote_public_key)`: Genera la clave compartida (`shared_key`) en bytes.

Conversión a Base64: Se presenta en pantalla (`shared_b64`).

KDF (SHA-256) PARA DERIVAR CLAVE AES 256

```
1 def derivar_clave_aes256():
2
3     print("\nIngresa la clave ECDH (Base64):")
4     ecdh_b64 = input("> ")
5
6     try:
7         ecdh_bytes = base64.b64decode(ecdh_b64.strip())
8     except Exception:
9         print("ERROR al decodificar base64 de la clave ECDH.")
10        return
11
12    # KDF: ConcatKDF con SHA256, para obtener 256 bits
13    # No usamos salt ni info adicional (podrias añadirlo en un sistema real).
14    ckdf = ConcatKDFHash(
15        algorithm=hashes.SHA256(),
16        length=32, # 32 bytes = 256 bits
17        otherinfo=None,
18        backend=default_backend()
19    )
20
21    aes_key = ckdf.derive(ecdh_bytes) # 32 bytes
22
23    aes_b64 = base64.b64encode(aes_key).decode('utf-8')
24    print("\n=== CLAVE DERIVADA (AES-256) EN BASE64 ===")
25    print(aes_b64)
26    print("=====")
27
28    opcion = input("\n¿Deseas guardar esta clave derivada en un archivo (aes_key.key)? (s/n): ")
29    if opcion.lower() == 's':
30        with open("aes_key.key", 'w') as f:
31            f.write(aes_b64)
32        print("Archivo 'aes_key.key' creado con éxito.")
```



- Entrada de la clave ECDH: El usuario pega la clave ECDH que se generó anteriormente (en Base64).
- Decodificación: Se convierte la cadena Base64 a bytes reales.
- Derivación de clave:
Se crea un objeto ConcatKDFHash usando SHA256.
length=32 indica que queremos 32 bytes (256 bits) como resultado de la derivación.
Finalmente, ckdf.derive(ecdh_bytes) produce la clave derivada.
- Conversión de la clave derivada a Base64: Se imprime la clave AES (256 bits) en Base64..

Esta función sirve para obtener una clave AES de 256 bits a partir de la clave ECDH cruda, de modo que luego podamos cifrar y descifrar archivos con AES-GCM.



CIFRAR ARCHIVO USANDO AES-GCM

```
1 def cifrar_archivo():
2
3     print("\n=== CIFRAR ARCHIVO CON AES-GCM ===")
4     aes_b64 = input("Ingresa la clave AES-256 (Base64): ").strip()
5
6     try:
7         aes_key = base64.b64decode(aes_b64)
8     except Exception:
9         print("ERROR al decodificar la clave AES en base64.")
10        return
11
12    if len(aes_key) != 32:
13        print("ERROR: la clave no es de 256 bits.")
14        return
15
16    nombre_in = input("Ingresa el nombre del archivo que deseas cifrar: ").strip()
17    if not os.path.exists(nombre_in):
18        print(f"ERROR: el archivo '{nombre_in}' no existe.")
19        return
20
21    # Leer contenido
22    with open(nombre_in, 'rb') as f:
23        plaintext = f.read()
24
25    # Crear AESGCM
26    aesgcm = AESGCM(aes_key)
27    import os
28    nonce = os.urandom(12) # GCM típicamente usa 12 bytes de nonce
29
30    # Cifrar
31    ciphertext = aesgcm.encrypt(nonce, plaintext, associated_data=None)
32    # El tag va embebido en el ciphertext al final, pero cryptography separa a la hora de desencriptar.
33    # En este caso, lo dejaremos tal cual, pues la librería lo maneja internamente.
34
35    # Extraer tag: cryptography no lo separa fácilmente, sino que lo hace al descifrar.
36    # No necesitamos guardarlo por separado, podemos re-cifrar cada vez. PERO
37    # si lo queremos explícito: tag = ciphertext[-16:], cipher_sin_tag = ciphertext[:-16]
38    # Sin embargo, la práctica normal es guardar ciphertext completo.
39
40    # Guardar nonce, ciphertext, tag (si se quiere) en Base64
41    nonce_b64 = base64.b64encode(nonce).decode('utf-8')
42    # Tag se puede extraer si queremos mostrarlo aparte:
43    tag = ciphertext[-16:]
44    tag_b64 = base64.b64encode(tag).decode('utf-8')
45    # El verdadero ciphertext "puro" sin tag sería:
46    real_cipher = ciphertext[:-16]
47    real_cipher_b64 = base64.b64encode(real_cipher).decode('utf-8')
48
49    # Preguntar por nombre de salida
50    nombre_out = input("Nombre del archivo de salida (p.ej. 'cifrado.gcm')?: ").strip()
51    if not nombre_out:
52        nombre_out = "cifrado.gcm"
53
54    # Guardar en 3 líneas: nonce, tag, cipher
55    with open(nombre_out, 'w') as f:
56        f.write(nonce_b64 + "\n")
57        f.write(tag_b64 + "\n")
58        f.write(real_cipher_b64 + "\n")
59
60    print(f"\nArchivo cifrado creado: {nombre_out}")
61    print("Contiene 3 líneas en Base64: nonce, tag, ciphertext.\n")
62
```



Instituto Politécnico Nacional

Escuela Superior de Cómputo
Selected Topics in Cryptography

